

Human-in-the-loop DEvops Report

CI/CD

Okay, this is a comprehensive request! We'll create a GitHub Actions workflow that: 1. **Triggers** on push to `main` and pull requests. 2. **Lints and Tests** the Python Flask application. 3. **Builds** a Docker image for the Flask application. 4. **Pushes** the Docker image to a container registry (e.g., Docker Hub, GitHub Container Registry). 5. **Deploys** the updated application to a Kubernetes cluster. --- ## 1. Project Structure (Expected) Before we dive into the workflow, make sure your project has a structure similar to this: `my-flask-app/` `.github/` `workflows/` `ci-cd.yml` # This is our GitHub Actions workflow `app/` `__init__.py` `main.py` # Your Flask application entry point `templates/` `index.html` `tests/` `test_app.py` # Your Python unit tests `requirements.txt` # Python dependencies `Dockerfile` # Docker build instructions `k8s/` `deployment.yaml` # Kubernetes Deployment manifest `service.yaml` # Kubernetes Service manifest `ingress.yaml` # (Optional) Kubernetes Ingress manifest `.dockerignore` `.gitignore` --- ## 2. GitHub Secrets You will need to set up the following secrets in your GitHub repository (`Settings > Secrets > Actions`): **DOCKER_USERNAME**: Your Docker Hub username or the username for your container registry. **DOCKER_PASSWORD**: Your Docker Hub access token (recommended over password) or password for your container registry. **DOCKER_REGISTRY**: The full URL of your container registry (e.g., `docker.io` for Docker Hub, `ghcr.io` for GitHub Container Registry). **KUBE_CONFIG_DATA**: The base64-encoded content of your Kubernetes cluster's `kubeconfig` file. **How to get it**: 1. Get your `kubeconfig` file (usually from your cloud provider or by setting up `kubectl` context). 2. Run: `cat ~/.kube/config | base64` (on Linux/macOS) or `Get-Content ~/.kube/config | Format-Base64` (on PowerShell). 3. Copy the output and paste it as the secret value. **KUBE_NAMESPACE**: The Kubernetes namespace where you want to deploy your application (e.g., `default`, `production`, `staging`). **K8S_DEPLOYMENT_NAME**: The name of your Kubernetes Deployment resource (e.g., `flask-app-deployment`). This will be used to update the image. --- ## 3. `Dockerfile` Example `dockerfile` # Use an official Python runtime as a parent image FROM python:3.9-slim-buster # Set the working directory in the container WORKDIR /app # Install any needed packages specified in requirements.txt COPY requirements.txt . RUN pip install --no-cache-dir -r requirements.txt # Copy the rest of the application code COPY . . # Expose the port your Flask app runs on EXPOSE 5000 # Define environment variable ENV FLASK_APP=app/main.py ENV FLASK_RUN_HOST=0.0.0.0 # Run the Flask application using Gunicorn (recommended for production) # Install gunicorn in requirements.txt: gunicorn==20.1.0 CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:5000", "app.main:app"] # Or, if you prefer flask run (less robust for production): # CMD ["flask", "run"] --- ## 4. `requirements.txt` Example `Flask==2.2.2 gunicorn==20.1.0 pytest==7.2.1 flake8==6.0.0` --- ## 5. Kubernetes Manifests Example (`k8s/deployment.yaml`, `k8s/service.yaml`) **k8s/deployment.yaml** `apiVersion: apps/v1 kind: Deployment metadata: name: flask-app-deployment # <-- This should match your K8S_DEPLOYMENT_NAME secret labels: app: flask-app spec: replicas: 2 selector: matchLabels: app: flask-app template: metadata: labels: app: flask-app spec: containers: - name: flask-app # This image will be updated by GitHub Actions image: your_docker_registry/your_flask_app_image:latest ports: - containerPort: 5000 resources: limits: memory: "128Mi" cpu: "500m" env: - name: FLASK_ENV value: production # Add probes for readiness and liveness livenessProbe: httpGet: path: /health port: 5000 initialDelaySeconds: 15 periodSeconds: 20 readinessProbe: httpGet: path: /health port: 5000 initialDelaySeconds: 5 periodSeconds: 10` **k8s/service.yaml** `apiVersion: v1 kind: Service metadata: name: flask-app-service labels: app: flask-app spec: selector: app: flask-app ports: - protocol: TCP port: 80 targetPort: 5000 type: LoadBalancer # Use ClusterIP if you have an Ingress controller` --- ## 6. GitHub Actions Workflow (`.github/workflows/ci-cd.yml`) `name: Python Flask App CI/CD with Docker and Kubernetes on: push: branches: - main pull_request: branches: - main workflow_dispatch: # Allows manual triggering of the workflow env: # Common variables for the workflow IMAGE_NAME: my-flask-app # Name of your Docker image K8S_PATH: k8s/ # Path to`

your Kubernetes manifests PYTHON_VERSION: 3.9 jobs: test: name: Lint and Test Python Application runs-on: ubuntu-latest steps: - name: Checkout repository uses: actions/checkout@v3 - name: Set up Python `{{ env.PYTHON_VERSION }}` uses: actions/setup-python@v4 with: python-version: `{{ env.PYTHON_VERSION }}` - name: Install Python dependencies run: | python -m pip install --upgrade pip pip install -r requirements.txt pip install flake8 pytest # Ensure testing tools are installed - name: Lint with Flake8 run: | flake8 . --count --select=E9,F63,F7,F82 --show-source --statistics flake8 . --count --exit-zero --max-complexity=10 --max-line-length=127 --statistics - name: Run Pytest tests run: | pytest tests/ build-and-push-docker: name: Build and Push Docker Image runs-on: ubuntu-latest needs: test # This job depends on the 'test' job succeeding if: github.ref == 'refs/heads/main' # Only run on push to main for deployment steps: - name: Checkout repository uses: actions/checkout@v3 - name: Set up Docker BuildX uses: docker/setup-buildx-action@v2 - name: Log in to Docker Registry uses: docker/login-action@v2 with: registry: `{{ secrets.DOCKER_REGISTRY }}` username: `{{ secrets.DOCKER_USERNAME }}` password: `{{ secrets.DOCKER_PASSWORD }}` - name: Build and Push Docker image uses: docker/build-push-action@v4 with: context: . file: ./Dockerfile push: true tags: | `{{ secrets.DOCKER_REGISTRY }}`/`{{ env.IMAGE_NAME }}`:latest `{{ secrets.DOCKER_REGISTRY }}`/`{{ env.IMAGE_NAME }}`:`{{ github.sha }}` # Unique tag for this commit deploy-to-kubernetes: name: Deploy to Kubernetes runs-on: ubuntu-latest needs: build-and-push-docker # This job depends on the 'build-and-push-docker' job succeeding if: github.ref == 'refs/heads/main' # Only run on push to main for deployment environment: production # Use GitHub Environments for protection rules steps: - name: Checkout repository uses: actions/checkout@v3 - name: Set up Kubeconfig run: | mkdir -p ~/.kube echo `"{{ secrets.KUBE_CONFIG_DATA }}"` | base64 -d > ~/.kube/config chmod 600 ~/.kube/config echo "Kubeconfig set up successfully." - name: Install kubectl uses: azure/setup-kubectl@v3 # Using Azure's action, but works for any K8s cluster - name: Apply Kubernetes manifests run: | kubectl config use-context docker-desktop # Replace if you use specific context kubectl apply -f `{{ env.K8S_PATH }}` -n `{{ secrets.KUBE_NAMESPACE }}` echo "Kubernetes manifests applied." - name: Update Deployment image run: | kubectl set image deployment/`{{ secrets.K8S_DEPLOYMENT_NAME }}` `{{ env.IMAGE_NAME }}`=`{{ secrets.DOCKER_REGISTRY }}`/`{{ env.IMAGE_NAME }}`:`{{ github.sha }}` -n `{{ secrets.KUBE_NAMESPACE }}` echo "Deployment image updated to `{{ secrets.DOCKER_REGISTRY }}`/`{{ env.IMAGE_NAME }}`:`{{ github.sha }}`" - name: Verify Deployment Rollout run: | kubectl rollout status deployment/`{{ secrets.K8S_DEPLOYMENT_NAME }}` -n `{{ secrets.KUBE_NAMESPACE }}` --timeout=5m echo "Deployment rollout verified." --- ## Explanation of the Workflow 1. `**name**`: A human-readable name for your workflow. 2. `**on**`: Defines when the workflow runs. * `push` to `main`: Triggers the full CI/CD pipeline. * `pull_request` to `main`: Triggers only the `test` job (for code quality checks before merge). * `workflow_dispatch`: Allows you to manually trigger the workflow from the GitHub Actions UI. 3. `**env**`: Global environment variables for the entire workflow. * `IMAGE_NAME`: The name you give to your Docker image. * `K8S_PATH`: The directory containing your Kubernetes manifests. * `PYTHON_VERSION`: The Python version to use for testing. 4. `**jobs**`: * `test` job: * `runs-on: ubuntu-latest`: Specifies the runner environment. * `steps`: * `actions/checkout@v3`: Checks out your repository code. * `actions/setup-python@v4`: Sets up the specified Python version. * `pip install -r requirements.txt`: Installs your application's Python dependencies. * `flake8` and `pytest`: Runs your linting and unit tests. * `build-and-push-docker` job: * `needs: test`: This job will only run if the `test` job passes. * `if: github.ref == 'refs/heads/main'`: Ensures this job only runs when pushing to the `main` branch. * `docker/setup-buildx-action@v2`: Configures Docker Buildx for enhanced build capabilities. * `docker/login-action@v2`: Logs into your specified Docker registry using GitHub Secrets. * `docker/build-push-action@v4`: Builds your Docker image and pushes it to the registry with two tags: `latest` and a unique tag based on the Git commit SHA (`github.sha`). * `deploy-to-kubernetes` job: * `needs: build-and-push-docker`: This job will only run if the Docker image is successfully built and pushed. * `if: github.ref == 'refs/heads/main'`: Ensures this job only runs when pushing to the `main` branch. * `environment: production`: This links to a GitHub Environment named "production". You can configure deployment protection rules for this environment (e.g., manual approval). * `Kubeconfig Setup`: Decodes the base64 `KUBE_CONFIG_DATA` secret into a `~/.kube/config` file, allowing `kubectl` to connect to your cluster. * `azure/setup-kubectl@v3`: Installs `kubectl` on the runner. * `kubectl apply -f ...`: Applies

all your Kubernetes manifest files from the `k8s/` directory. This creates/updates Services, Ingress, etc. * `kubectl set image ...`: This is the crucial step for updating the deployment. It specifically targets your `flask-app-deployment` and updates the image to the newly pushed one (using the `github.sha` tag). * `kubectl rollout status ...`: Waits for the deployment to successfully roll out, ensuring the new pods are running before the workflow finishes. --- This setup provides a robust CI/CD pipeline for your Flask application with Docker and Kubernetes, leveraging GitHub Actions best practices. Remember to replace placeholder values like `your\_docker\_registry` and `your\_flask\_app\_image` in your `k8s/deployment.yaml`.